

PROJECT DOCUMENTATION

Motor Spares Inventory & Sales Management System

A desktop application for inventory, point-of-sale, customer and supplier management

Prepared by: Mthabisi E. Vundla

Document Version: 1.0

Date: August 2026

Status: Draft for Development

Table of Contents

Table of Contents.....	2
Document Control	4
Revision History	4
Purpose of This Document	4
1. Introduction	5
1.1 Purpose	5
1.2 Project Background.....	5
1.3 Scope	5
1.4 Intended Audience	5
2. Development Methodology.....	6
2.1 Chosen Methodology: Incremental / Iterative Development	6
2.2 Justification	6
2.3 Development Lifecycle.....	6
3. Requirements Analysis	7
3.1 Stakeholders / Actors	7
3.2 Functional Requirements.....	7
3.3 Non-Functional Requirements.....	7
3.4 Business Rules.....	8
3.5 Validation Rules	8
3.6 Numbering Conventions	9
3.7 Error Handling.....	9
4. System Analysis.....	10
4.1 Use Case Diagram	10
4.2 Key Use Case Descriptions	10
4.3 Data Flow Diagram (Level 0)	11
5. System Design.....	12
5.1 System Architecture	12
Layer Responsibilities	12
5.2 Database Design — Entity-Relationship Diagram.....	12
5.3 Data Dictionary	13
Part	13
StockMovement.....	13
Sale	14
SaleItem	14
Customer	14
Supplier	14
PurchaseOrder	15

User.....	15
AuditLog.....	15
5.3.1 Key Constraints & Indexes	16
5.4 Receipt Specification	17
5.5 User Interface Design	18
Navigation Flow	18
6. Technology Stack	19
7. Development Environment Setup	20
8. Project Structure.....	21
9. Implementation Plan	22
Phase 1 — Environment & Database Foundation (Week 1).....	22
Phase 2 — Authentication & User Management (Week 2).....	22
Phase 3 — Inventory Module (Weeks 3–4)	22
Phase 4 — POS / Sales Module (Weeks 5–6).....	22
Phase 5 — Customers & Suppliers (Week 7)	22
Phase 6 — Reporting (Week 8).....	22
Phase 7 — Testing & Hardening (Week 9).....	23
Phase 8 — Packaging & Deployment (Week 10)	23
10. Testing Strategy	24
10.1 Levels of Testing	24
10.2 Sample Test Cases.....	24
11. Deployment Plan	25
11.1 Packaging	25
11.2 Installation Steps	25
11.3 Backup Strategy	25
12. Risk Management.....	26
13. Maintenance & Future Enhancements.....	27
13.1 Maintenance.....	27
13.2 Planned Future Enhancements (Deferred from v1 Scope).....	27
Appendix A: Glossary.....	28
Appendix B: Sample Data	29
Sample Part.....	29
Sample Supplier	29
Sample Customer.....	29
Sample Sale.....	29

Document Control

Revision History

Version	Date	Author	Description
1.0	August 2026	M. E. Vundla	Initial documentation for development handoff
1.1	August 2026	M. E. Vundla	Added business rules, validation rules, numbering conventions, error handling, receipt specification, database constraints, minimal audit log, and sample data. Added Stock Adjustment and Void Sale requirements (FR-15, FR-16, FR-17).

Purpose of This Document

This document is the single reference for building the Motor Spares Inventory & Sales Management System from the ground up. It covers requirements, design, architecture, database schema, the technology stack, a phased implementation plan, testing strategy, and deployment steps, so that a developer can pick it up and build the system end to end without needing outside clarification.

1. Introduction

1.1 Purpose

The purpose of this project is to design and build a desktop software system for a motor spares (auto parts) retail business. The system will replace manual, paper-based stock and sales tracking with a single application covering inventory management, point-of-sale (POS) transactions, customer records, supplier records, and business reporting.

1.2 Project Background

The business currently sells auto parts to walk-in customers and mechanics from a single shop location. Stock levels, sales, and supplier orders are not currently tracked in a unified digital system, which makes it difficult to know what is in stock, what is selling, and what needs to be reordered. This project addresses that gap with a purpose-built system for the shop counter.

1.3 Scope

In scope for the initial release:

- Inventory management for auto parts (add, edit, search, stock levels)
- Point-of-sale transactions with automatic stock deduction
- Customer records and purchase history
- Supplier records and purchase orders
- Sales and profit reporting
- Local data storage that works without an internet connection

Out of scope for the initial release (deferred to a later phase):

- Remote / cloud access and multi-branch synchronisation
- Mobile companion app
- Online store / e-commerce integration
- Barcode scanner hardware integration (designed for, not built, in v1)

1.4 Intended Audience

This document is written for the developer(s) building the system. It assumes basic familiarity with software development but no prior context on this specific project — every stage needed to go from an empty folder to a working, deployed system is covered below.

2. Development Methodology

2.1 Chosen Methodology: Incremental / Iterative Development

This project will be built using an incremental (iterative) development approach, influenced by Agile principles, rather than a strict Waterfall model. The system is broken into working increments — each one a usable slice of functionality — built, tested, and reviewed in sequence, with each increment adding onto a stable base.

2.2 Justification

- The project is being built by a single developer for a real, operating business — a working system is needed early, not only at the very end.
- Requirements may shift slightly once the shop owner sees and uses early increments (e.g. what fields matter on a receipt, how discounts are applied).
- Each module (inventory, POS, customers, suppliers, reports) is reasonably self-contained, which suits incremental delivery well.
- A pure Waterfall approach risks discovering usability problems only after the whole system is built.

2.3 Development Lifecycle

Each increment passes through the same six stages shown below, before the next increment begins:

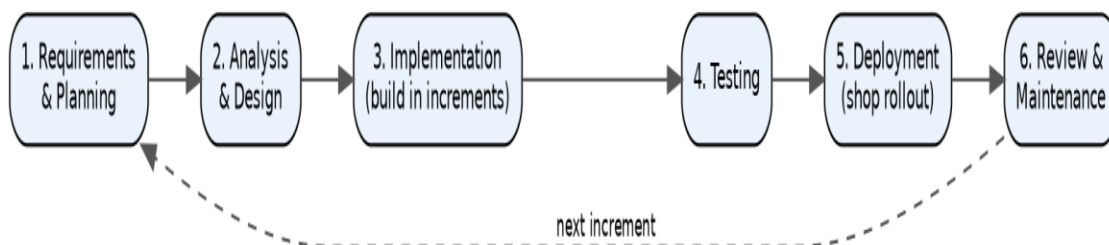


Figure 2.1 — Incremental development lifecycle used for each phase

3. Requirements Analysis

3.1 Stakeholders / Actors

Actor	Description
Cashier	Front-counter staff who process sales, look up parts, and print receipts.
Owner / Admin	Shop owner or manager — full access, including inventory, suppliers, users, and reports.

3.2 Functional Requirements

ID	Requirement
FR-01	The system shall allow an authorised user to log in with a username and password.
FR-02	The system shall allow adding, editing, and deactivating parts in inventory.
FR-03	The system shall allow searching parts by part number, name, category, or compatible vehicle.
FR-04	The system shall record a sale, deduct sold quantities from stock automatically, and generate a receipt.
FR-05	The system shall support at least cash and EcoCash as payment methods on a sale.
FR-06	The system shall flag parts that fall at or below their reorder level.
FR-07	The system shall allow recording stock received from a supplier (stock-in).
FR-08	The system shall maintain customer records with name, phone number, and purchase history.
FR-09	The system shall maintain supplier records and allow creating purchase orders.
FR-10	The system shall generate sales reports (daily, weekly, custom date range).
FR-11	The system shall generate a profit-margin report per part and per category.
FR-12	The system shall support at least two user roles: Cashier and Owner/Admin, with different permissions.
FR-13	The system shall allow the Owner/Admin to back up the database to a local file.
FR-14	The system shall print or export a receipt for each completed sale.
FR-15	The system shall allow the Owner/Admin to make a manual stock adjustment (e.g. after a physical stock take), recording the reason.
FR-16	The system shall allow a cashier to void or cancel a sale before it is finalised, with no effect on stock or records.
FR-17	The system shall record an audit entry whenever a stock adjustment, price change, or part deactivation occurs, capturing the user, timestamp, and old/new values.

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Common actions (search, add to cart, checkout) shall complete in under 1 second on typical shop hardware.
NFR-02	Availability	The system shall function fully offline; no feature required for a sale shall depend on internet access.
NFR-03	Usability	A cashier with basic computer literacy shall be able to complete a sale within a short, one-time walkthrough.
NFR-04	Reliability	The database shall not lose data on an ungraceful shutdown (e.g. power cut) mid-transaction.
NFR-05	Security	Passwords shall be stored hashed, never in plain text.
NFR-06	Maintainability	Code shall be organised into clear modules (UI, business logic, data access) to support future changes.
NFR-07	Portability	The system shall be packaged so it can be installed on a Windows shop PC without requiring a manual Python install.

3.4 Business Rules

These rules govern system behaviour in situations the functional requirements alone don't fully specify. They exist to remove guesswork during implementation.

ID	Rule
BR-01	Stock quantity can never go below zero. A sale cannot be completed if it would take any part's quantity_on_hand negative.
BR-02	A part with existing sales history cannot be permanently deleted — it can only be deactivated (hidden from search, kept for reporting).
BR-03	Only an Admin user may change a part's cost_price. Cashiers can view but not edit prices.
BR-04	Selling price must be greater than or equal to cost price, unless an Admin explicitly overrides it (e.g. clearance stock).
BR-05	Every stock movement (in or out) must be linked to a reason — Sale, Purchase, Adjustment, Damage, or Return.
BR-06	A sale is only finalised (and stock deducted) once payment is confirmed at checkout — items in the cart before that point do not affect stock.
BR-07	A voided/cancelled sale (FR-16) has no effect on inventory or reports — it is discarded, not recorded as a zero-value sale.
BR-08	A user account cannot be deleted if it has processed sales — it is deactivated instead, so sales history keeps a valid cashier reference.

3.5 Validation Rules

Field	Rule
Part — selling_price	Must be greater than 0

Field	Rule
Part — cost_price	Must be greater than or equal to 0
Part — part_number	Required; must be unique
Part — name	Required
Part/Sale — quantity	Must be greater than 0; cannot exceed quantity_on_hand at checkout
Customer — phone	Optional — a sale can proceed without a customer record
Customer/Supplier — name	Required
Stock-in	Must reference a valid supplier before it can be saved
User — username	Required; must be unique
User — password	Minimum 6 characters (adjust to shop policy at build time)

3.6 Numbering Conventions

Fixed formats for human-readable reference numbers, so the developer doesn't have to invent them ad hoc:

Entity	Format	Example
Part number	Free text (shop/manufacturer part number as printed on the part)	OF-001
Receipt number	REC-YYYYMMDD-NNNN (sequence resets daily)	REC-20260801-0015
Purchase order	PO-YYYYMMDD-NNNN (sequence resets daily)	PO-20260801-0002
Customer reference	CUS-NNNNN (zero-padded, sequential)	CUS-00102

3.7 Error Handling

Expected failure scenarios and how the system should respond — these map directly onto the test cases in Section 10.2.

Scenario	System Response
Duplicate part number entered	Reject the save; show an inline error naming the conflicting part.
Sale would take stock below zero	Block the sale for that line item; show available quantity.
Database file cannot be opened on launch	Show a clear recovery message; do not crash silently.
Wrong password entered	Reject login; after 5 consecutive failed attempts, lock the account for a short cooldown period.
Stock-in submitted with no supplier selected	Block save until a supplier is chosen.
App closed / power lost mid-sale	On next launch, the sale is either fully committed or fully absent — never partially applied (see NFR-04).

4. System Analysis

4.1 Use Case Diagram

The diagram below shows the two system actors and their main interactions with the system.

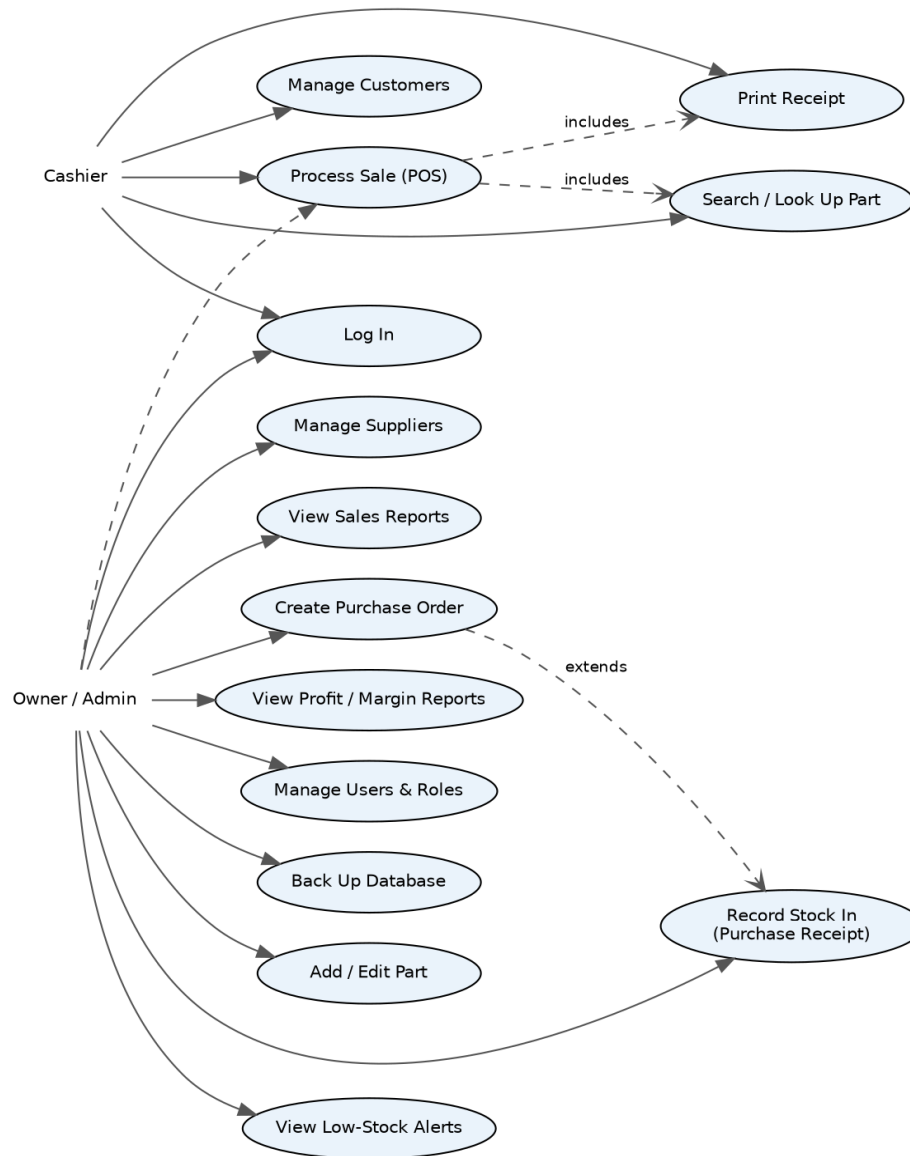


Figure 4.1 — Use case diagram

4.2 Key Use Case Descriptions

Use Case	Actor	Description
Process Sale (POS)	Cashier	Cashier searches/selects parts, system calculates total, records payment, deducts stock, prints receipt.
Add / Edit Part	Owner/Admin	Owner enters or updates a part's details, price, and stock levels in inventory.

Use Case	Actor	Description
Record Stock-In	Owner/Admin	Owner records parts received from a supplier, increasing stock on hand.
View Low-Stock Alerts	Owner/Admin	System highlights parts at or below their reorder level for restocking decisions.
Create Purchase Order	Owner/Admin	Owner generates an order to a supplier for specific parts and quantities.
View Reports	Owner/Admin	Owner views sales totals, best/worst sellers, and profit margins over a chosen date range.

4.3 Data Flow Diagram (Level 0)

The context-level data flow diagram below shows how data moves between external entities and the system as a whole.

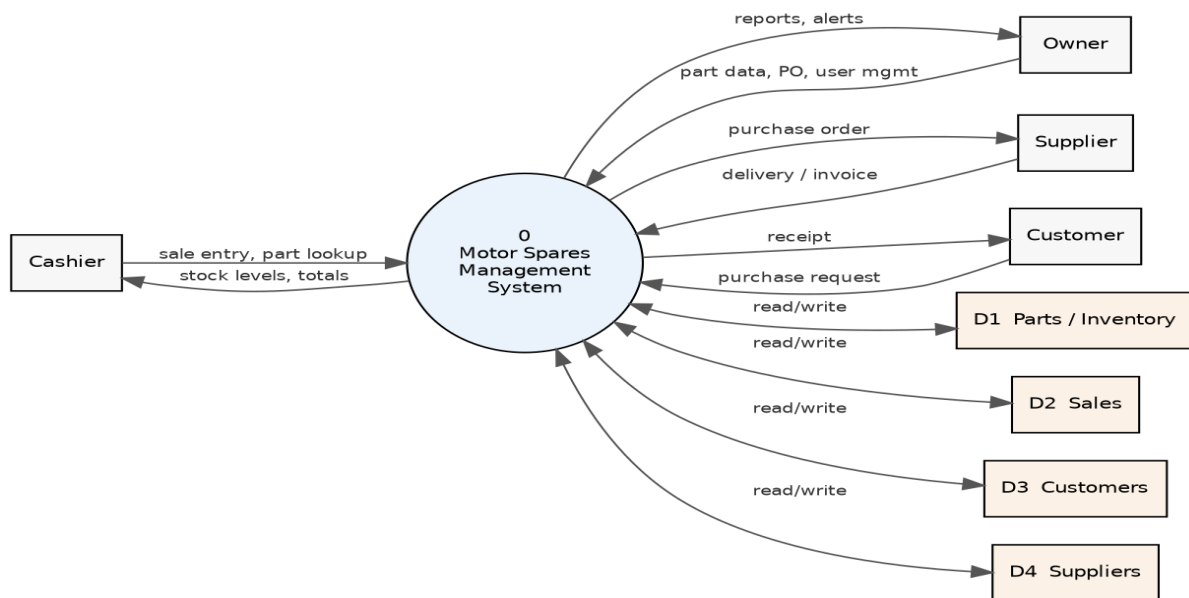


Figure 4.2 — Level 0 data flow diagram

5. System Design

5.1 System Architecture

The system follows a layered architecture, separating the user interface, business logic, and data storage. This keeps the UI code free of database logic and makes each layer easier to test and modify independently.

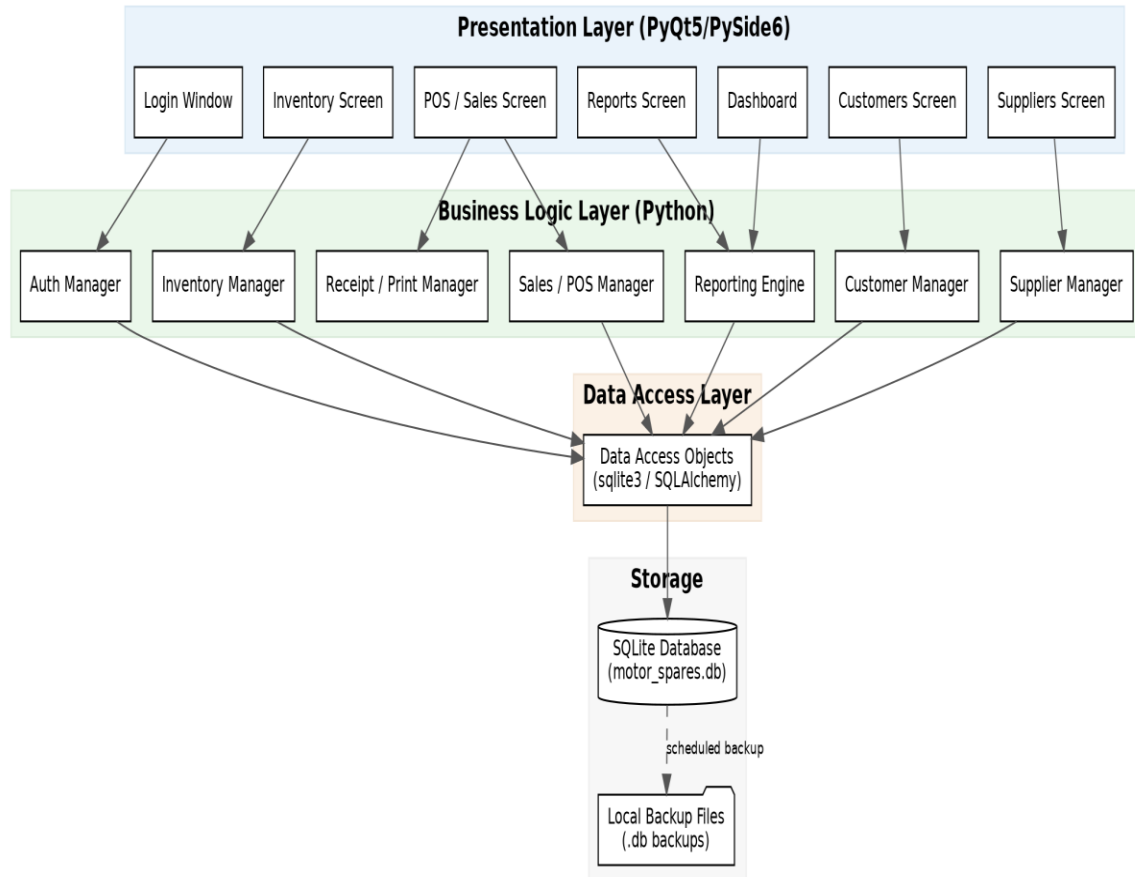


Figure 5.1 — Layered system architecture

Layer Responsibilities

Layer	Responsibility
Presentation	PyQt windows and widgets — Login, Dashboard, Inventory, POS, Customers, Suppliers, Reports.
Business Logic	Manager classes that enforce rules — e.g. cannot sell more stock than is on hand, cannot delete a part with sales history.
Data Access	Functions/classes that read and write to SQLite; the only layer that contains SQL.
Storage	The SQLite database file and periodic local backup files.

5.2 Database Design — Entity-Relationship Diagram

The database is designed around eight core entities. Relationships are shown below, followed by the full data dictionary for each table.

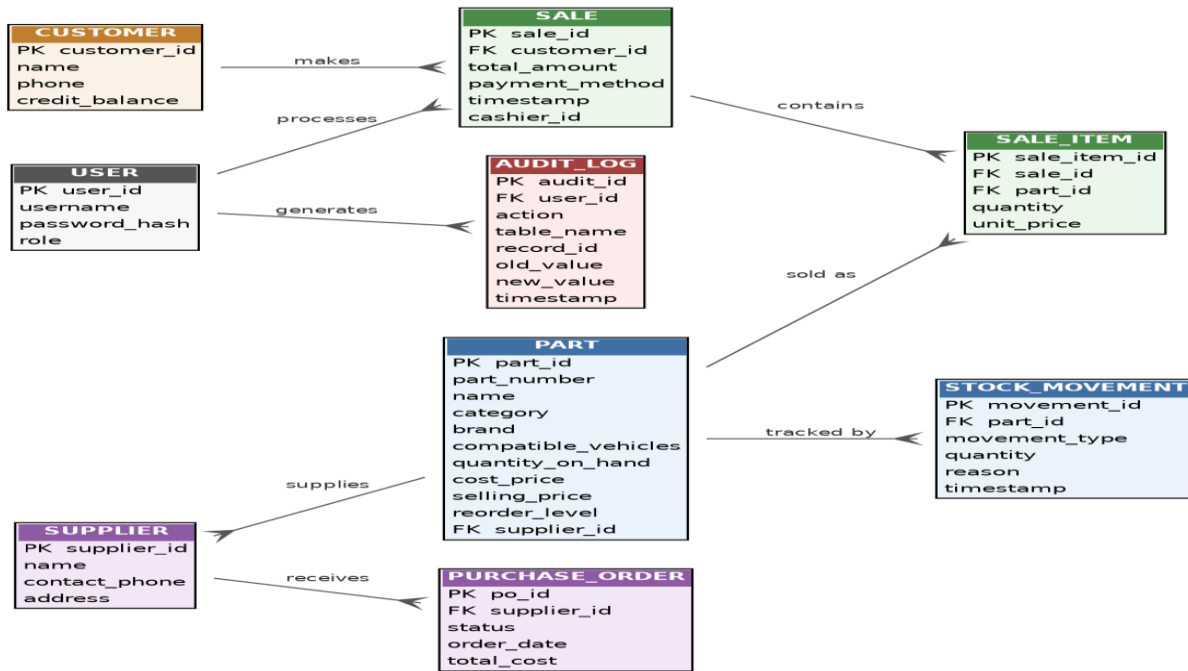


Figure 5.2 — Entity-relationship diagram

5.3 Data Dictionary

Part

Field	Type / Constraints	Description
part_id	INTEGER, PK, autoincrement	Unique identifier
part_number	TEXT, unique, not null	Manufacturer / shop part number
name	TEXT, not null	Descriptive name of the part
category	TEXT	e.g. Engine, Brakes, Suspension, Electrical
brand	TEXT	Manufacturer / brand name
compatible_vehicles	TEXT	Free text or structured make/model/year
quantity_on_hand	INTEGER, not null, default 0	Current stock level
cost_price	REAL, not null	Price paid to supplier
selling_price	REAL, not null	Price charged to customer
reorder_level	INTEGER, default 0	Threshold that triggers a low-stock flag
supplier_id	INTEGER, FK -> Supplier.supplier_id	Primary supplier for this part

StockMovement

Field	Type / Constraints	Description
movement_id	INTEGER, PK, autoincrement	Unique identifier
part_id	INTEGER, FK -> Part.part_id	Part affected
movement_type	TEXT, ('IN' 'OUT')	Direction of stock change
quantity	INTEGER, not null	Quantity moved
reason	TEXT	e.g. Sale, Purchase, Damage, Correction
timestamp	TEXT (ISO datetime), not null	When the movement occurred

Sale

Field	Type / Constraints	Description
sale_id	INTEGER, PK, autoincrement	Unique identifier
customer_id	INTEGER, FK -> Customer.customer_id, nullable	Null for walk-in / no record
total_amount	REAL, not null	Total value of the sale
payment_method	TEXT	Cash, EcoCash, etc.
timestamp	TEXT (ISO datetime), not null	When the sale occurred
cashier_id	INTEGER, FK -> User.user_id	User who processed the sale

SaleItem

Field	Type / Constraints	Description
sale_item_id	INTEGER, PK, autoincrement	Unique identifier
sale_id	INTEGER, FK -> Sale.sale_id	Parent sale
part_id	INTEGER, FK -> Part.part_id	Part sold
quantity	INTEGER, not null	Quantity sold
unit_price	REAL, not null	Price at time of sale

Customer

Field	Type / Constraints	Description
customer_id	INTEGER, PK, autoincrement	Unique identifier
name	TEXT, not null	Customer name
phone	TEXT	Contact number
credit_balance	REAL, default 0	Outstanding balance, if credit is extended

Supplier

Field	Type / Constraints	Description
supplier_id	INTEGER, PK, autoincrement	Unique identifier
name	TEXT, not null	Supplier / company name
contact_phone	TEXT	Contact number
address	TEXT	Physical or postal address

PurchaseOrder

Field	Type / Constraints	Description
po_id	INTEGER, PK, autoincrement	Unique identifier
supplier_id	INTEGER, FK -> Supplier.supplier_id	Supplier the order is placed with
status	TEXT, ('Draft' 'Ordered' 'Received')	Current order status
order_date	TEXT (ISO date)	Date order was placed
total_cost	REAL	Total cost of the order

User

Field	Type / Constraints	Description
user_id	INTEGER, PK, autoincrement	Unique identifier
username	TEXT, unique, not null	Login username
password_hash	TEXT, not null	Hashed password (never plain text)
role	TEXT, ('Cashier' 'Admin')	Determines permissions

AuditLog

Field	Type / Constraints	Description
audit_id	INTEGER, PK, autoincrement	Unique identifier
user_id	INTEGER, FK -> User.user_id	User who performed the action
action	TEXT, ('ADJUSTMENT' 'PRICE_CHANGE' 'DEACTIVATE')	Type of tracked event (FR-17)
table_name	TEXT, not null	Table the change was made to
record_id	INTEGER, not null	ID of the affected record
old_value	TEXT	Previous value (nullable for creations)
new_value	TEXT	New value
timestamp	TEXT (ISO datetime), not null	When the change occurred

5.3.1 Key Constraints & Indexes

These formalise the constraints implied by the data dictionary above and the business rules in Section 3.4:

Table	Constraint
Part	UNIQUE(part_number); CHECK(quantity_on_hand >= 0); CHECK(cost_price >= 0); CHECK(selling_price >= 0); INDEX on category, brand for search
Sale	CHECK(payment_method IN ('Cash','EcoCash','Card')); FK customer_id ON DELETE SET NULL; FK cashier_id ON DELETE RESTRICT
SaleItem	CHECK(quantity > 0); FK sale_id ON DELETE CASCADE; FK part_id ON DELETE RESTRICT
StockMovement	CHECK(movement_type IN ('IN','OUT')); FK part_id ON DELETE RESTRICT
User	UNIQUE(username); CHECK(role IN ('Cashier','Admin'))
PurchaseOrder	CHECK(status IN ('Draft','Ordered','Received')); FK supplier_id ON DELETE RESTRICT
AuditLog	FK user_id ON DELETE RESTRICT; INDEX on table_name, record_id for lookups

5.4 Receipt Specification

Every completed sale (FR-14) generates a receipt with the following contents, in order:

- Shop name and address (from configuration — see Section 8)
- Receipt number (format: REC-YYYYMMDD-NNNN — see Section 3.6)
- Date and time of sale
- Cashier name
- Customer name (if a customer record was attached to the sale)
- Line items: part name, quantity, unit price, line subtotal
- Subtotal, discount (if any — deferred to v1.1, see Section 13), grand total
- Payment method
- Footer message (e.g. "Thank you for your business" — configurable)

5.5 User Interface Design

The table below lists every screen in the system, its main UI elements, and its purpose. These map directly onto the PyQt windows built during implementation.

Screen	Key Elements	Purpose
Login Window	Username/password fields, login button	Authenticates the user and loads role-based permissions
Dashboard	Summary cards (today's sales, low-stock count), navigation menu	Landing screen after login; quick overview
Inventory Screen	Searchable/sortable parts table, Add/Edit/Delete buttons, low-stock filter	Manage the parts catalogue
POS / Sales Screen	Part search box, running cart, quantity controls, total, payment method, checkout button	Process a sale from start to receipt
Customers Screen	Customer list, search, add/edit, purchase history view	Manage customer records
Suppliers Screen	Supplier list, add/edit, linked purchase orders	Manage supplier records and orders
Reports Screen	Date range picker, report type selector, table/summary output, export button	View and export business reports
User Management Screen	User list (Admin only), add/edit user, role assignment	Manage cashier/admin accounts

Navigation Flow

Login → Dashboard → (Inventory | POS | Customers | Suppliers | Reports | User Management, based on role) — the Dashboard acts as the central hub, with a persistent side menu for moving between modules without returning to the Dashboard each time.

6. Technology Stack

Component	Technology	Purpose
Programming Language	Python 3.11+	Primary development language
GUI Framework	PyQt5 or PySide6	Desktop user interface
Database	SQLite 3 (via Python's built-in sqlite3 module)	Local, file-based relational storage — no server needed
ORM (optional)	SQLAlchemy	Optional layer over raw SQL for cleaner data access code
Packaging	PyInstaller	Bundles the app into a single Windows executable
Version Control	Git + GitHub	Source control and backup of the codebase
Testing	pytest	Unit and integration testing
Receipt / PDF Export	reportlab	Generating printable receipts and PDF reports
IDE	VS Code or PyCharm	Development environment

Note: cloud/remote sync, if added in a later phase, would introduce a backend API (e.g. FastAPI) and a hosted database — deliberately excluded from this version's stack per current scope.

7. Development Environment Setup

1. Install Python 3.11 or later from python.org, ensuring "Add Python to PATH" is checked during installation.
2. Install Git from git-scm.com for version control.
3. Install an IDE — VS Code (with the Python extension) or PyCharm Community Edition.
4. Create a project folder, e.g. `motor_spares_system`, and open a terminal inside it.
5. Create a virtual environment: `python -m venv venv`
6. Activate it — Windows: `venv\Scripts\activate` | macOS/Linux: `source venv/bin/activate`
7. Install dependencies: `pip install PyQt5 sqlalchemy reportlab pytest`
8. Initialise Git: `git init`, then create a `.gitignore` excluding `venv/`, `__pycache__`, and `*.db`
9. Create the initial folder structure (see Section 8).
10. Run a minimal PyQt "hello world" window to confirm the environment is working before writing real features.

8. Project Structure

The recommended folder layout mirrors the layered architecture from Section 5.1, keeping UI, business logic, and data access cleanly separated:

```

motor_spares_system/├─ main.py                # Application entry point ─ requirements.txt
# Python dependencies ─ database/ ──┬─ db_manager.py          # Connection handling, schema
creation ──┬─ motor_spares.db      # SQLite database file (generated) ─ models/ ──┬─ part.py
├─ sale.py ──┬─ customer.py ──┬─ supplier.py ──┬─ user.py ──┬─ purchase_order.py ──
managers/      # Business logic layer ──┬─ auth_manager.py ──┬─ inventory_manager.py
├─ sales_manager.py ──┬─ customer_manager.py ──┬─ supplier_manager.py ──┬─
report_manager.py ──┬─ ui/                # Presentation layer (PyQt windows) ──┬─
login_window.py ──┬─ dashboard.py ──┬─ inventory_screen.py ──┬─ pos_screen.py ──┬─
customers_screen.py ──┬─ suppliers_screen.py ──┬─ reports_screen.py ──┬─
user_management_screen.py ──┬─ utils/ ──┬─ receipt_generator.py ──┬─ backup.py ──┬─ tests/ ──┬─
├─ test_inventory_manager.py ──┬─ test_sales_manager.py ──┬─ test_db_manager.py ──┬─ logs/ ──┬─
├─ app.log                    # Runtime error/event log (generated) ──┬─ assets/ ──┬─ icons, logo, etc.

```

9. Implementation Plan

The build is broken into eight sequential phases. Each phase follows the development lifecycle in Section 2.3 and produces a working, demonstrable deliverable before the next phase begins.

Phase 1 — Environment & Database Foundation (Week 1)

- Set up development environment (Section 7) and project structure (Section 8)
- Design and create the SQLite schema for all eight tables (Section 5.3)
- Build the data access layer (`db_manager.py`) with connection handling and CRUD helper functions
- Set up basic logging using Python's built-in logging module (writing to `logs/app.log`) in place of `print()` statements, so errors after deployment are traceable
- Write a basic PyQt main window that launches successfully

Deliverable: A running application shell with a working, empty database.

Phase 2 — Authentication & User Management (Week 2)

- Build the User model and `auth_manager.py` (login, password hashing)
- Build the Login window
- Build basic role-based access control (Cashier vs Admin)
- Build the User Management screen (Admin only)

Deliverable: Users can log in and are routed based on role.

Phase 3 — Inventory Module (Weeks 3–4)

- Build the Part model and `inventory_manager.py`
- Build the Inventory screen: list, search, add, edit, deactivate parts
- Implement low-stock flagging based on `reorder_level`
- Implement stock-in recording (linked to StockMovement)

Deliverable: Inventory can be fully managed through the UI.

Phase 4 — POS / Sales Module (Weeks 5–6)

- Build the Sale and SaleItem models and `sales_manager.py`
- Build the POS screen: part search, cart, totals, payment method selection
- Implement automatic stock deduction on checkout
- Implement receipt generation (reportlab) and printing

Deliverable: A cashier can complete a full sale from search to printed receipt.

Phase 5 — Customers & Suppliers (Week 7)

- Build Customer and Supplier models and their managers
- Build the Customers screen (list, add/edit, purchase history)
- Build the Suppliers screen and Purchase Order creation flow

Deliverable: Customer and supplier records are fully manageable, linked to sales and stock-in.

Phase 6 — Reporting (Week 8)

- Build report_manager.py — sales totals, best/worst sellers, profit margins
- Build the Reports screen with date-range filtering
- Add export to PDF/CSV

Deliverable: Owner can view and export business reports for any date range.

Phase 7 — Testing & Hardening (Week 9)

- Write unit tests for all manager classes (Section 10)
- Perform integration and system testing across modules
- Fix bugs found; test edge cases (e.g. selling more than available stock)
- Implement local backup functionality

Deliverable: A stable, tested build ready for real-world use.

Phase 8 — Packaging & Deployment (Week 10)

- Package the app with PyInstaller into a standalone executable
- Install and test on the actual shop PC
- Load real opening inventory data
- Train the shop owner / cashier on daily use

Deliverable: The system is live and in daily use at the shop.

10. Testing Strategy

10.1 Levels of Testing

Level	Focus	Tooling	When
Unit Testing	Individual functions/classes (e.g. <code>inventory_manager.add_part()</code>)	pytest	Throughout each phase, as code is written
Integration Testing	Interactions between layers (e.g. POS screen -> <code>sales_manager</code> -> database)	pytest + manual	End of each phase
System Testing	The complete application against the full requirements list (Section 3)	Manual, scripted test cases	Phase 7
User Acceptance Testing (UAT)	Real usage by the shop owner/cashier against real scenarios	Manual, guided walkthrough	End of Phase 7 / start of Phase 8

10.2 Sample Test Cases

ID	Title	Steps	Expected Result	Requirement
TC-01	Sell more than available stock	Attempt to add 10 units of a part with 5 in stock to the cart	System blocks the action / warns insufficient stock	FR-04
TC-02	Low-stock flag	Set a part's quantity below its reorder level	Part appears on the low-stock list	FR-06
TC-03	Login with wrong password	Enter a valid username with an incorrect password	Login is rejected with a clear error message	FR-01
TC-04	Complete a sale	Add parts to cart, select payment method, checkout	Stock decreases, sale is recorded, receipt is generated	FR-04, FR-14
TC-05	Cashier permissions	Log in as Cashier and attempt to open User Management	Access is denied / option is hidden	FR-12

11. Deployment Plan

11.1 Packaging

The application will be packaged into a standalone Windows executable using PyInstaller, so the shop PC does not need Python installed manually:

```
pyinstaller --onefile --windowed --icon=assets/icon.ico main.py
```

11.2 Installation Steps

1. Copy the packaged executable and an empty (or pre-seeded) database file to the shop PC.
2. Create a desktop shortcut for the shop owner / cashier.
3. Load the real opening inventory (initial parts, quantities, prices) into the system.
4. Create user accounts for the owner and cashier(s).
5. Perform a short walkthrough / training session covering login, a sample sale, and adding stock.
6. Set up a simple backup routine (see 11.3).

11.3 Backup Strategy

Since the system runs fully offline on local storage, the database file itself is the single point of failure. The system includes a manual "Back Up Database" action (FR-13) that copies the .db file with a timestamp to a backups/ folder. It is recommended the shop owner also periodically copies this backup folder to a USB drive.

12. Risk Management

Risk	Likelihood	Impact	Mitigation
Power outage during a transaction	Medium	High	SQLite writes are transactional; test ungraceful shutdown scenarios in Phase 7 (NFR-04).
Data loss (no cloud backup in v1)	Medium	High	Manual local backup feature (FR-13) plus periodic USB backup by the owner.
Developer unfamiliar with Python at start	High	Medium	Learn-while-building approach; start with small, guided increments (Phase 1).
Scope creep (owner requests cloud/mobile early)	Medium	Medium	Scope is fixed for v1 (Section 1.3); track new requests for a v2 phase.
Incorrect stock counts from manual data entry	Medium	Medium	Add a stock-take / adjustment feature; validate entries at the UI layer.

13. Maintenance & Future Enhancements

13.1 Maintenance

Once deployed, the system should be reviewed periodically (e.g. monthly for the first quarter) to fix any issues found in real use and to confirm backups are being taken.

13.2 Planned Future Enhancements (Deferred from v1 Scope)

- Cloud / remote sync — allow the owner to view reports and stock remotely from a phone or browser.
- Barcode scanner integration for faster part lookup at checkout.
- Multi-branch support if the business expands to a second location.
- Mobile companion app for on-the-go stock checks.
- Customer credit / layaway tracking with automated statements.
- SMS/WhatsApp notifications to customers or the owner (e.g. low-stock alerts).
- Discounts — per line item or whole-sale, once the core POS flow is proven.
- Returns / refunds with automatic stock restoration.
- Opening cash float and end-of-day cash reconciliation.
- CSV import/export for bulk-loading or exporting inventory data.
- Full audit system covering all entities (v1 ships with a minimal audit log — see Section 5.3 — scoped to stock adjustments, price changes, and deactivations only).
- Configuration file (shop name, receipt footer, currency, logo path) instead of hard-coded values.
- Expanded project structure separating services/ from repositories/ if the codebase grows large enough to warrant it.
- Formal coding standards document (PEP8/Black/ruff enforcement) — reasonable once collaborating with another developer; not a blocker for a solo build.

Appendix A: Glossary

Term	Definition
POS	Point of Sale — the screen/process used to record a customer transaction.
SKU / Part Number	Unique code identifying a specific part.
Reorder Level	The stock quantity at which a part should be restocked.
SQLite	A lightweight, file-based relational database requiring no separate server.
PyQt / PySide	Python libraries for building desktop graphical user interfaces.
ORM	Object-Relational Mapper — code that lets a program interact with a database using objects instead of raw SQL.
UAT	User Acceptance Testing — testing performed by the actual end user (shop owner/cashier) before go-live.
ERD	Entity-Relationship Diagram — a diagram showing database tables and how they relate.

Appendix B: Sample Data

Worked examples to sanity-check the data dictionary and to use as seed data during development and testing.

Sample Part

Field	Value
part_number	OF-001
name	Oil Filter
brand	Toyota
category	Engine
compatible_vehicles	Toyota Hilux 2015-2020
quantity_on_hand	35
cost_price	\$5.00
selling_price	\$8.00
reorder_level	10
supplier_id	-> ABC Parts (see below)

Sample Supplier

Field	Value
name	ABC Parts
contact_phone	+263 77 000 0000
address	Bulawayo, Zimbabwe

Sample Customer

Field	Value
customer_id (reference)	CUS-00102
name	T. Moyo
phone	+263 77 111 2222

Sample Sale

Field	Value
receipt_number (reference)	REC-20260801-0015
items	2x Oil Filter (OF-001) @ \$8.00 = \$16.00
total_amount	\$16.00
payment_method	EcoCash

